

Data sharing

- ◆ *Threads* of a process *share* process data
 - ◆ E.g., multiple threads read/write same data
 - ◆ Consider banking process
 - ◆ Thread 1: withdraw money from account
 - ◆ Thread 2: deposit money into account
 - Account is *shared* by threads 1 and 2
- ◆ Similarly, two different *processes* may share data
 - ◆ E.g., output data of one process is input to another process

*The next few concepts will be explained/
demonstrated with data sharing among multiple
threads of same process, but the concepts apply
equally to data sharing among different processes*

Banking process operation

Account balance

100

Thread 1:

Load balance

Subtract withdrawal amount

Store updated balance

Thread 2:

Load balance

Add deposit amount

Store updated balance

Banking process operation

Account balance

100

Thread 1:



Load balance

Subtract withdrawal amount

Store updated balance

Thread 2:

100

Load balance

Add deposit amount

Store updated balance

Banking process operation

Account balance

100

Thread 1:



Load balance

Subtract withdrawal amount

Store updated balance

Thread 2:

150

Load balance

Add deposit amount

Store updated balance

Banking process operation

Account balance

100

Thread 1:

100

Load balance

Subtract withdrawal amount

Store updated balance

Thread 2:

150

Load balance

Add deposit amount

Store updated balance

Banking process operation

Account balance

100

Thread 1:

50

Load balance

Subtract withdrawal amount

Store updated balance

Thread 2:

150

Load balance

Add deposit amount

Store updated balance

Banking process operation

Account balance

50

Thread 1:

50

Load balance

Subtract withdrawal amount

Store updated balance

Thread 2:

150

Load balance

Add deposit amount

Store updated balance

Banking process operation

Account balance

150

Balance should have been 100!

Thread 1:

50

Load balance

Subtract withdrawal amount

Store updated balance

Thread 2:

150

Load balance

Add deposit amount

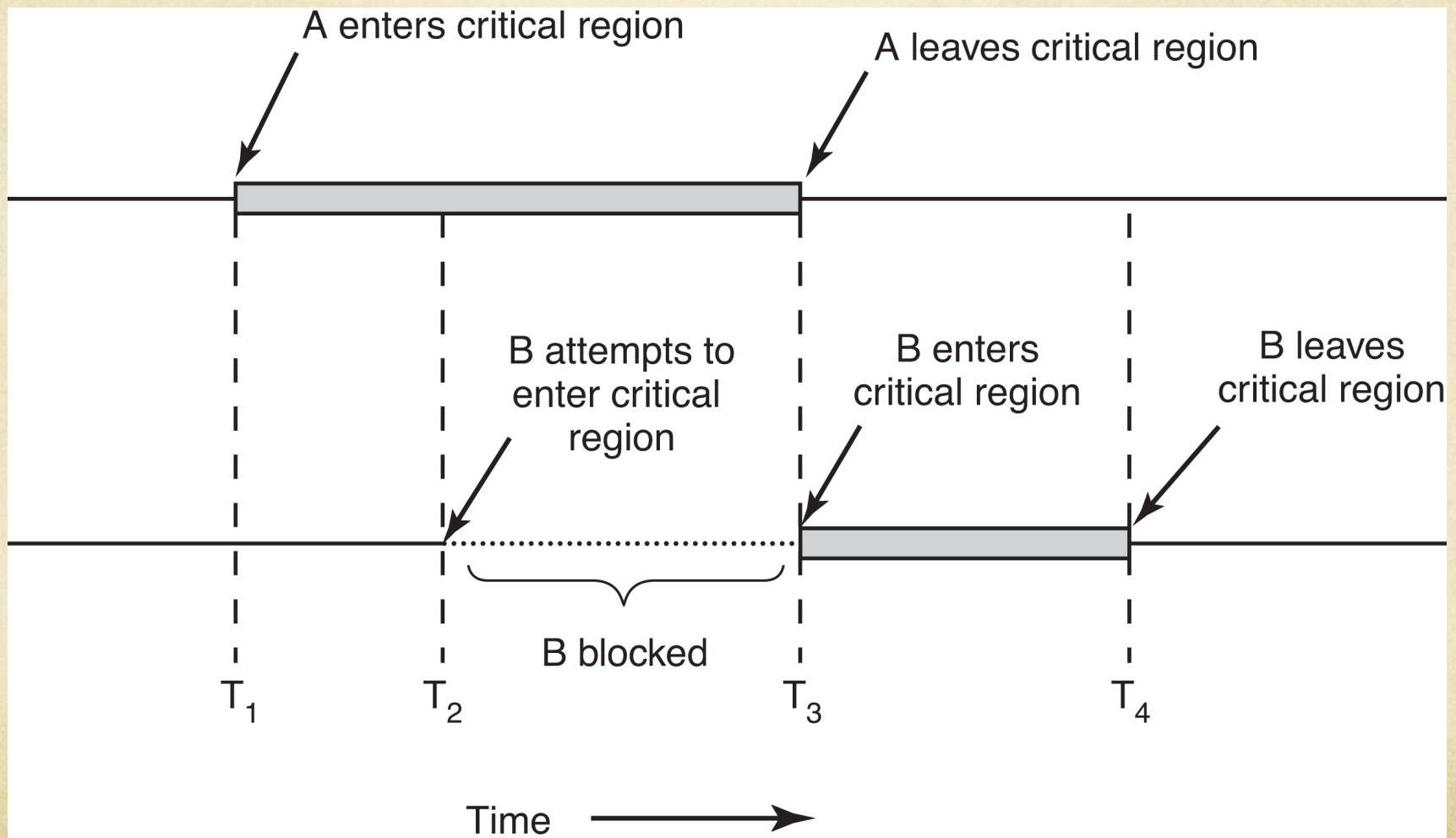
Store updated balance

Such a scenario in which final result depends on order of execution is called *race condition*

Understanding *race conditions*

- ◆ Threads must not be *interleaved* during access to *shared* data
- ◆ This property is called **mutual exclusion** among threads
- ◆ Region of code during which shared data is accessed
 - ◆ *Critical section/critical region*
- ◆ *No two threads may simultaneously be inside critical sections accessing same shared data*

What we need...



This rule avoids race conditions...

- ◆ There are more rules in place from system perspective
 - ◆ Must consider fairness, efficiency, etc.
- ◆ Overall *mutual exclusion requirements* are
 - ◆ *No two threads/processes may simultaneously be in critical sections accessing same shared data*
 - ◆ *No assumptions must be made about hardware (#procs, speed, etc.)*
 - ◆ *A thread/process not in critical section must not prevent other thread/process from entering critical section*
 - ◆ *No thread/process should have to wait for ever to enter critical section*